影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

# 第 5 章 空間域影像強化

## 本章學習目標

- 區分點運算與鄰域運算，說明空間域強化的基本框架與它在全書中的定位。
- 運用負片、對數與 Gamma 校正等灰階轉換調整影像明暗與對比，並理解 LUT 最佳化。
- 讀懂直方圖，並以直方圖等化與 CLAHE 改善對比不足的影像。
- 正確定義卷積與相關，說明濾波器核、邊界處理與可分離濾波的運算量差異。
- 比較均值、高斯、中值、雙邊四種平滑濾波的特性與適用雜訊型態。
- 以 Laplacian、Unsharp Masking 與 High-Boost 進行影像銳化，並理解其代價。
- 說明多影格合成的原理，並理解計算攝影如何以軟體突破硬體限制。

## 叫獸開講|手機如何在黑夜裡看見：夜景模式的祕密

2018 年 11 月，Google 為 Pixel 手機推出 Night Sight(夜視模式)。它的能力在當時令人難以置信：在照度低至 0.3 勒克斯的環境——那是連人類視覺都已退化成模糊單色的暗度——不用腳架、不用閃光燈，手持就能拍出乾淨、銳利、色彩正確的照片。負責這項技術的是 Marc Levoy，史丹佛大學計算攝影的先驅、後來成為 Google 的傑出工程師。

低光攝影為什麼困難?Levoy 團隊在技術文件中解釋得很清楚：手機的鏡頭與感光元件都很小，收集到的光子本來就少;而主要的雜訊來源是進入鏡頭的光子數量本身的隨機波動(散粒雜訊，第 3、7 章)，加上電荷轉數字時的讀取雜訊，兩者

共同拉低訊噪比。物理學給的解方很單純：訊噪比隨曝光時間的平方根上升，曝光愈久畫面愈乾淨。但問題是——手持拍照，你會抖;被拍的人，也在動。長曝光換來的是一張明亮的糊照。

Google 的答案不是換更大的鏡頭，而是換一種思路：連拍一疊短曝光影像，在軟體中對齊後合併。一張張「不夠亮但夠銳利」的照片疊起來，隨機的雜訊互相抵消，亮度與訊噪比同時提升。這個「以多換好」的想法，正是本章 5.6 節要展開的多影格合成。

Night Sight 把這套做法推到極致，其中幾個設計極具巧思。其一是運動測光(motion metering)：在你按下快門之前，系統就已經在量測手抖程度與場景中的運動量，據此預測接下來的運動並自動決定每張的曝光時間——場景夠靜時，單張曝光甚至可超過 300 毫秒。其二是影格數的動態調整：若偵測到手機放在腳架上就拍 6 張，手持則最多 15 張，並限制總擷取時間以免使用者失去耐心。其三是穩健合併：採用專為高雜訊影像設計的分塊對齊策略，能丟棄運動過大的區塊，避免鬼影。其四是學習式白平衡——因為暗處的光源色溫難以判斷，他們訓練了專門針對夜景的演算法(第 8 章會回到這個主題)。

最後一個設計尤其耐人尋味：色調映射刻意受到繪畫的啟發。團隊發現，若只是單純把夜景調亮，結果會像大白天拍的，失去夜的氛圍。因此他們仿效畫家描繪夜景的手法——提高對比、把陰影壓到近乎全黑、讓畫面周圍沉入黑暗。這是一個很好的提醒：影像強化的目標，未必是「還原真實」，有時是「傳達感受」。這正是本章「強化」與第 7 章「還原」的根本差異。

拆開來看，Night Sight 用到的技術幾乎全在本章：多張平均去雜訊(5.6 節)、對比拉伸與色調曲線(5.1 節)、局部對比增強(5.2 節)，而對齊則用到第 6 章的相位相關與第 13 章的特徵匹配。學完這一章，你就有能力自己寫一個「窮人版夜景模式」——這正是 Lab 5-4 的任務。

思考題：「連拍多張再平均」為什麼能降低雜訊?如果雜訊是隨機且零均值的，平均 N 張後雜訊的標準差會變成原來的幾分之一?(提示：與叫獸開講中「訊噪比隨曝光時間平方根上升」是同一個數學原理)

## 5.1 空間域處理的框架與灰階轉換

### 5.1.1 本章的定位

空間域處理是指直接對像素數值進行運算，相對於第 6 章在頻率域的處理。它可寫成 g(x, y) = T[f(x, y)]，其中 f 是輸入影像、g 是輸出影像、T 是作用在 (x, y) 鄰域上的運算子。當鄰域只有一個像素時，T 退化為灰階轉換函數，稱為點運算(Point Operation)：輸出只取決於該像素自己的值，與鄰居無關。當鄰域擴大到 3×3、5×5 時，就成為 5.3 節的空間濾波(鄰域運算)。

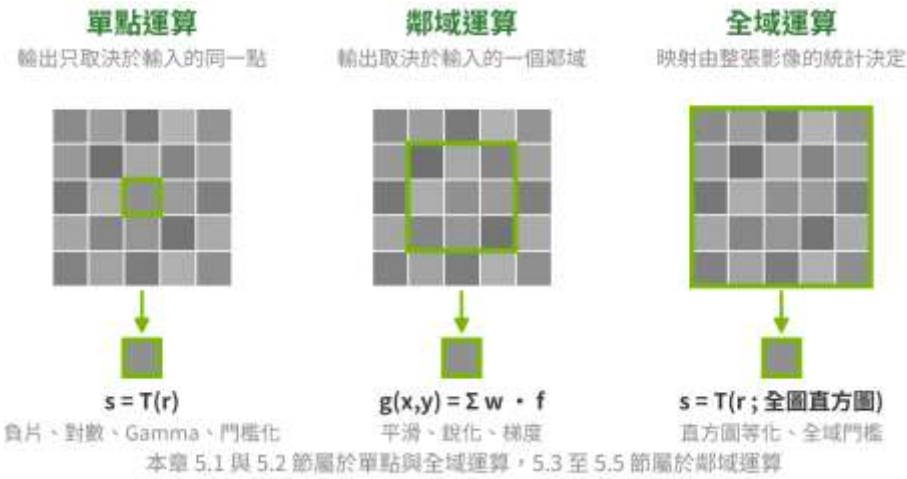



*圖 5-1　空間域處理的三種作用範圍*

這裡也要釐清本章與第 7 章的關係，因為兩者常被混淆。本章談的是影像強化(Enhancement)：目標是讓影像「看起來更好」或「更適合後續處理」，判準相當主觀——就像叫獸開講中 Night Sight 刻意把陰影壓黑以營造夜的氛圍，那並非還原真實，卻是更好的照片。第 7 章的影像還原(Restoration)則是客觀的逆問題：先建立劣化的數學模型，再反推最接近原始的估計。一句話總結：強化是化妝，還原是手術。

### 5.1.2 負片與對數轉換

負片轉換 s = L - 1 - r(8 位元時為 255 - r)把黑白顛倒。它在醫學影像中特別有用：X 光片上細小的白色病灶，反轉成黑色後在淺色背景中反而更醒目——這利用了人眼對暗背景上的亮物與亮背景上的暗物，敏感度並不相同的特性。

對數轉換 s = c·log(1 + r) 則大幅拉伸暗部、壓縮亮部，適合處理動態範圍極大的影像。最經典的例子是第 6 章的傅立葉頻譜：直流分量的數值往往比其他成分大好幾個數量級，不取對數幾乎只看得到中央一個亮點。當你在第 6 章看到「顯示頻譜前要取對數」時，用的就是這裡的工具。

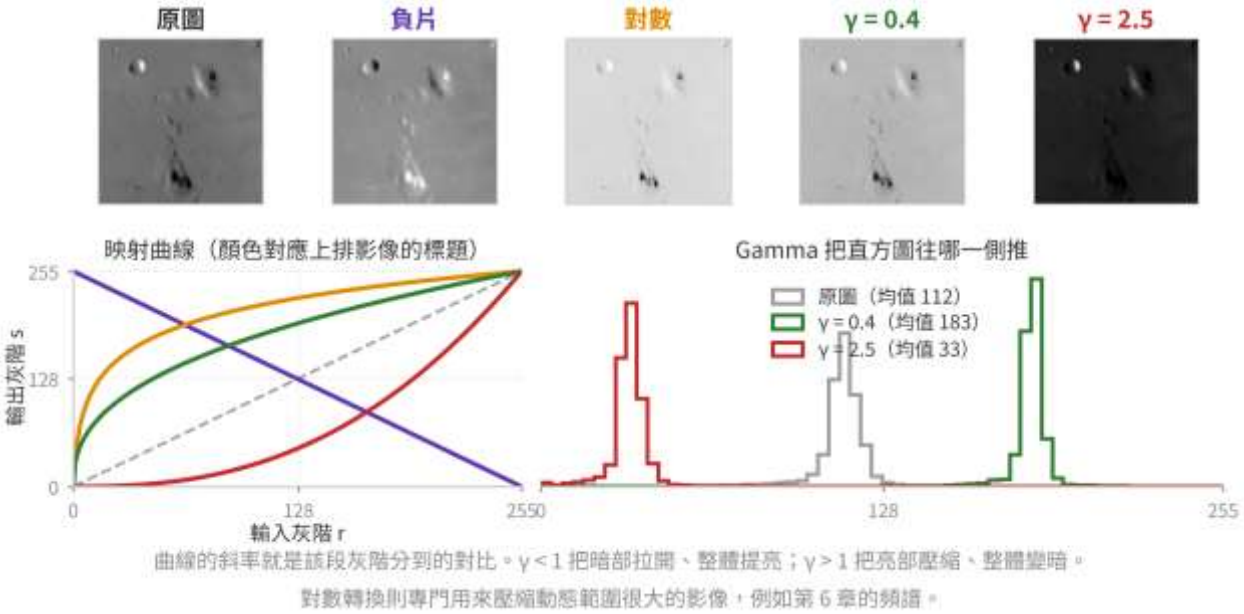



*圖 5-2　四種灰階轉換的映射曲線與效果*

## 5.1.3 Gamma 校正與冪次法則

冪次法則轉換 s = c·r^γ 是實務上最常用的明暗調整工具。γ 小於 1 時，暗部被拉亮(整體變亮、暗部細節浮現);γ 大於 1 時，暗部被壓暗(整體變暗、亮部細節保留)。

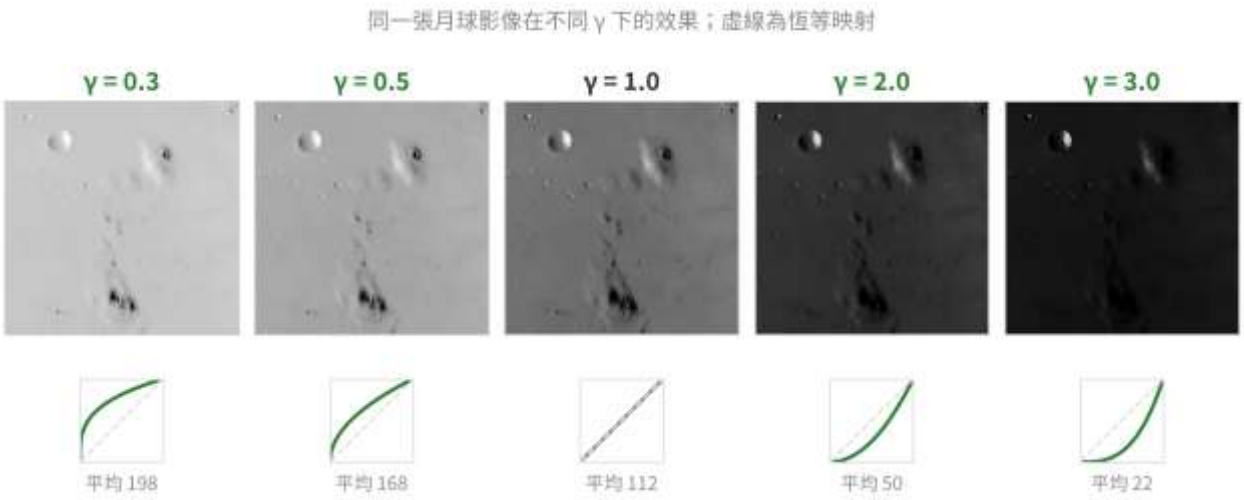



*圖 5-3　Gamma 值對影像的影響*

它的重要性不只在美觀，還有一段硬體的歷史淵源：早期映像管螢幕的亮度響應天生近似 γ = 2.2 的冪次曲線，因此影像在儲存時預先做了反向的 Gamma 編碼來抵消。這就是為什麼 8 位元 JPEG 的像素值並非線性正比於光強度，而第 3 章提到的 RAW 檔保留的才是線性感測值。

這個差別在做精確的光度量測時會咬人：若你要判斷「這個區域的實際亮度是那個區域的幾倍」，直接用 JPEG 的像素值相除會得到錯誤答案，必須先做反 Gamma 轉換回到線性空間。這也是第 7 章影像還原、第 8 章白平衡計算時要留意的細節——許多演算法在推導時都假設輸入是線性的。

**程式 5-1 灰階轉換：負片、Gamma、對數**

```
import cv2
import numpy as np

img = cv2.imread("dark_room.jpg")

# --- 負片 ---
negative = 255 - img

# --- Gamma 校正:用查表(LUT)一次算好 256 個對應值,最快 ---
def gamma_correct(img, gamma):
    inv = 1.0 / gamma
    table = np.array([((i / 255.0) ** inv) * 255
                      for i in range(256)]).astype("uint8")
    return cv2.LUT(img, table)

brighter = gamma_correct(img, 2.2)   # gamma>1 → 提亮暗部
darker   = gamma_correct(img, 0.45)  # gamma<1 → 壓暗
```

```
# --- 對數轉換(常用於顯示頻譜,第 6 章) ---
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)
log_img = 255 * np.log1p(gray) / np.log1p(255)
log_img = log_img.astype(np.uint8)

# --- 反 Gamma:回到線性空間再做光度計算(第 7、8 章需要) ---
linear = np.power(img.astype(np.float32) / 255.0, 2.2)
```

請注意程式中的 cv2.LUT()：灰階轉換只有 256 種可能的輸入，先建好查表再一次對映，遠比對每個像素重算冪次快——這正是第 2 章「向量化思維」的延伸，也是影像處理最常見的最佳化手法之一。在第 17 章的 Jetson 或 ESP32 等邊緣裝置上，這類最佳化尤其值得。

## 5.2 直方圖處理

### 5.2.1 讀懂直方圖

影像直方圖統計每個灰階值出現的像素個數，是影像的「亮度身分證」。曝光不足的影像，直方圖擠在左側;過曝擠在右側;對比不足則集中在中間一小段;對比良好的影像，分布通常較為寬廣。




*圖 5-4　四種曝光狀況的影像與直方圖*

直方圖是影像分析的第一個診斷工具，它的用途貫穿全書：攝影師看它判斷曝光;工程師據此決定該做什麼處理;第 7 章用平坦區域的直方圖形狀來辨識雜訊型態(鐘形是

高斯、兩端尖峰是椒鹽);第 9 章的 Otsu 門檻分割更是直接建立在直方圖之上——它假設前景與背景各形成一個峰，並自動找出最佳的分界點。養成「拿到影像先看直方圖」的習慣，你會少走很多彎路。

### 5.2.2 直方圖等化

直方圖等化(Histogram Equalization)的目標是讓灰階分布盡可能均勻，以擴展對比。其核心是把累積分布函數(CDF)當作轉換函數：先統計各灰階的機率 $p(r_k) = n_k / MN$，再計算累積機率並乘上 $L-1$ 得到映射值 $s_k = (L-1)\cdot\Sigma\, p(r_j)$。

直觀理解：某個灰階若聚集大量像素，CDF 在該處陡升，映射後鄰近灰階被拉開，對比自然增強;像素稀少的區段則被壓縮。換句話說，等化是把「有限的灰階資源」分配給「像素多的地方」——這與第 8 章色度降取樣、第 11 章 JPEG 量化把資源分配給人眼在意之處，是同一種資源分配的思維。

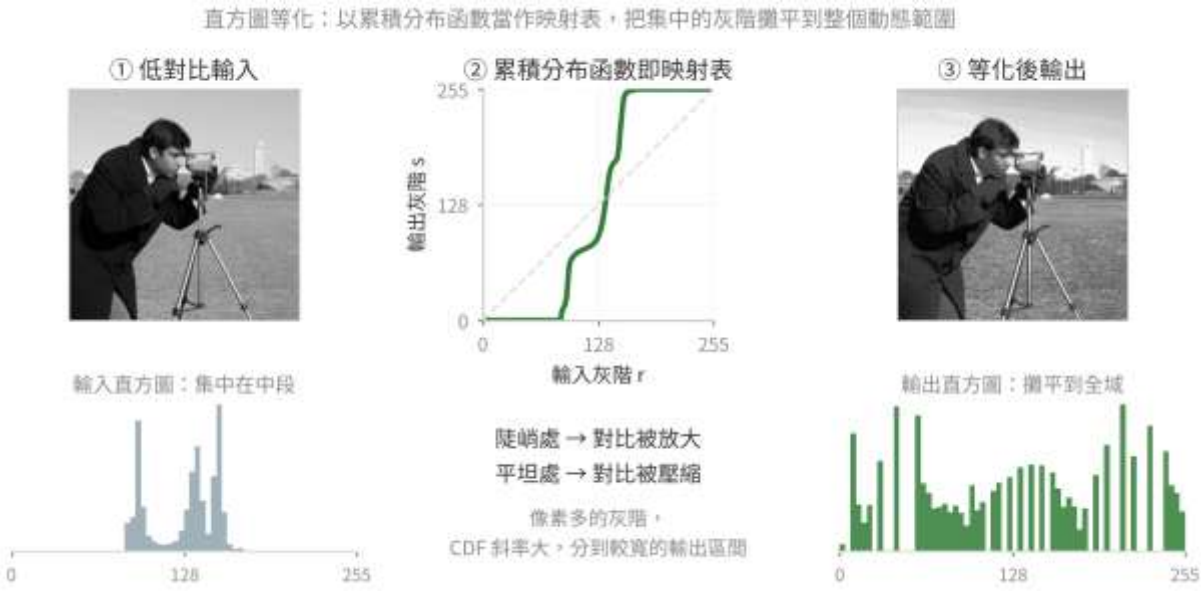



*圖 5-5　直方圖等化的三個步驟*

手算範例($L = 8$，共 64 像素)：設灰階 0 至 7 的像素數為 8、10、16、14、8、5、2、1。累積機率依序為 0.125、0.281、0.531、0.750、0.875、0.953、0.984、1.000;乘以 7 並四捨五入，得到映射 0→1、1→2、2→4、3→5、4→6、5→7、6→7、7→7。原本擠在中低灰階的像素被推展到整個範圍，對比明顯提升——這就是等化的

全部祕密。

但等化也有代價：它是「全自動、不可控」的，而且會放大平坦區域的雜訊(因為雜訊造成的微小灰階差異也被拉開了)。此外，映射後某些原本不同的灰階可能合併成同一階(如上例的 5、6、7 都映射到 7)，造成細節喪失。這解釋了為什麼實務上很少直接用全域等化，而多採用下一節的 CLAHE。

### 5.2.3 CLAHE：局部才是王道

全域等化有個致命弱點：它用整張影像的統計量決定映射。若影像同時有明亮窗戶與陰暗室內，等化結果往往顧此失彼——這正是第 8 章「洋裝效應」與第 9 章打光不均問題在灰階世界的翻版。

CLAHE(對比度受限的自適應直方圖等化)的解法有二：其一，把影像切成小格(例如 8×8)，各自等化後再以雙線性內插消除格線接縫，如此亮區與暗區各自依自己的統計量調整;其二，設定對比限制(clipLimit)，把直方圖中過高的長條「削頂」並把超出的部分均攤到其他灰階，避免雜訊被暴力放大——這個「削頂」正是 CLAHE 名稱中 Contrast Limited 的由來，也是它優於一般自適應等化的關鍵。

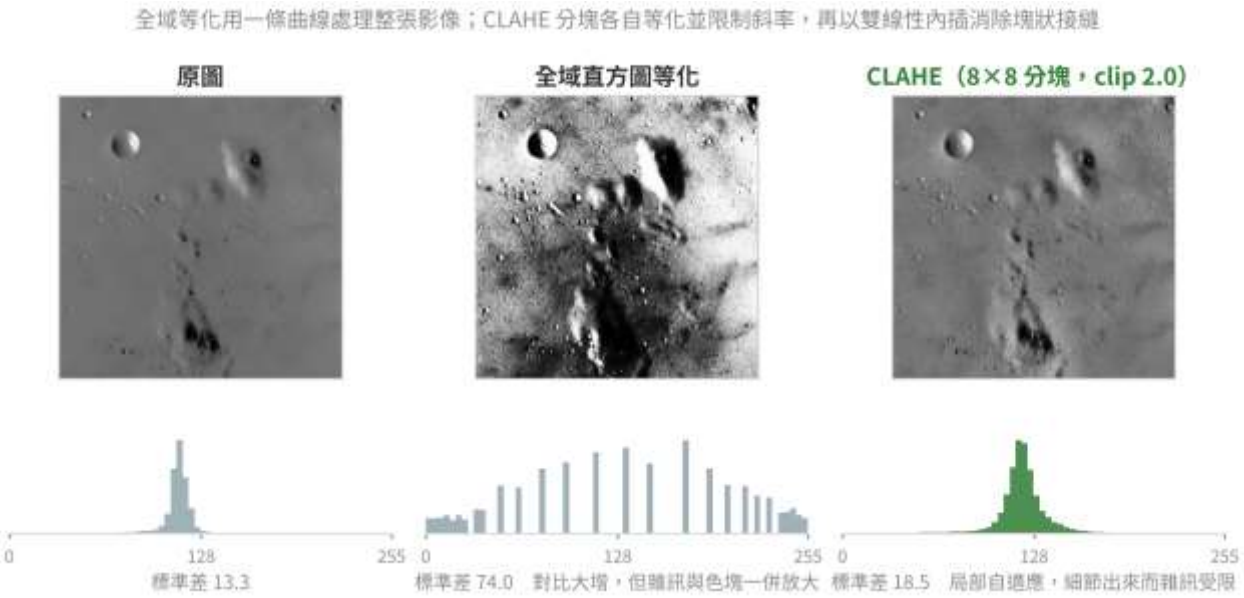



*圖 5-6　全域直方圖等化與 CLAHE 的比較*

實務上，CLAHE 幾乎總是比全域等化好用，是內視鏡影像、AOI 檢測、夜間監控

的常客。本書後續多章的前處理(第 9 章分割前、第 16 章訓練資料準備)都會用到它。

**程式 5-2 直方圖診斷、等化與 CLAHE**

```
import cv2, numpy as np

gray = cv2.imread("lowcontrast.jpg", cv2.IMREAD_GRAYSCALE)

# 先看直方圖:診斷是第一步
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
print("最亮/最暗像素值:", gray.max(), gray.min())
print("集中區間佔比:", (hist[80:160].sum() / gray.size))

eq = cv2.equalizeHist(gray)                    # 全域等化

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
cl = clahe.apply(gray)                         # 局部等化(建議)

# --- 彩色影像的正確做法:只等化亮度通道,別碰色彩 ---
bgr = cv2.imread("lowcontrast_color.jpg")
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)     # L=亮度, a,b=色彩
l, a, b = cv2.split(lab)
l = clahe.apply(l)
result = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)

# 錯誤示範:對 BGR 三通道分別等化 → 色偏
wrong = cv2.merge([cv2.equalizeHist(c) for c in cv2.split(bgr)])
```

程式最後兩段是常見考點：若對 BGR 三個通道分別等化，各通道的映射函數不同，色彩比例會被破壞，產生詭異的色偏。正確做法是轉到亮度與色彩分離的色彩空間(LAB 或 YCbCr)，只處理亮度通道——第 8 章會完整說明這些色彩空間的設計理由。

### 5.2.4 推導：為什麼累積分布函數剛好是對的映射

上一小節說「用累積分布函數當映射表」，但為什麼偏偏是它?這件事在連續的情形下可以嚴格證明，值得花兩段把它看清楚，因為同一套「以累積分布做變數變換」的手法，在第 14 章的機率校正與第 15 章的資料正規化中還會再出現。

設輸入灰階 r 已正規化到 [0, 1]，其機率密度為 p(r)。定義映射 $s = T(r) = \int_0^r p(w)\,dw$，也就是 r 的累積分布函數。由微積分基本定理，$dT/dr = p(r)$。再由機率密度的變數變換公式，輸出的密度為 $q(s) = p(r) \cdot |dr/ds| = p(r) / p(r) = 1$，對所有 $s \in [0, 1]$ 都成立——輸出正是均勻分布。換句話說，不論輸入的直方圖長什麼樣，只要拿它自己

的累積分布去映射，輸出的直方圖就是平的。本書以四十萬個樣本的 Beta 分布實測驗證：映射後二十個等寬區間的佔比全為 0.0500，與理論值分毫不差。

但數位影像是離散的，這個完美的結論會打折扣，原因有二。其一，灰階只有 256 階，映射後必須四捨五入回整數，原本不同的輸入灰階可能被捨入到同一個輸出灰階——合併之後就再也分不開，所以輸出的直方圖只會「盡量平」而不可能全平。其二，由此可推得一個常被忽略的性質：等化後影像的相異灰階數，永遠不會超過等化前的相異灰階數。等化能拉開對比，卻不會無中生有地創造出新的灰階層次——這也是為什麼對一張本來就只有少數幾階的影像做等化，結果會是刺眼的色塊而不是柔順的漸層。

## 5.3 空間濾波：卷積與相關

### 5.3.1 定義與差異

空間濾波把一個小矩陣(稱為濾波器核、遮罩或 kernel)滑過影像，在每個位置與鄰域對應相乘後加總，作為輸出像素值。




*圖 5-7　空間濾波的逐格計算*

這裡有兩個容易混淆的名詞：相關(Correlation)是直接對應相乘相加;卷積(Convolution)則先把核旋轉 180 度再做相同運算。對於對稱核(如高斯)，兩者結果完全相同，所以日常常被混用;但對非對稱核就會差一個翻轉。理論上要求卷積的原因是

它滿足交換律與結合律，並對應第 6 章的卷積定理。實務提醒：OpenCV 的 cv2.filter2D 執行的其實是相關，需要真正卷積時得自己先翻轉核。




*圖 5-8　相關與卷積的差別：核的 180° 翻轉*

一個值得先建立的連結：第 15 章的卷積神經網路，其「卷積層」做的正是這件事——差別在於，本章的核是人設計的(高斯核、Sobel 核)，而 CNN 的核是從資料中學出來的。當你在第 15 章看到 Conv2D 時，請記得它的數學核心就在這一節。

## 5.3.2 邊界處理

核滑到影像邊緣時，一部分會伸出界外，必須決定「界外的值是什麼」。常見策略：補零(BORDER_CONSTANT，簡單但邊緣會變暗)、複製邊緣像素(BORDER_REPLICATE)、鏡射(BORDER_REFLECT,OpenCV 預設之一，通常視覺效果最自然)、環繞(BORDER_WRAP)。



*圖 5-9　五種邊界處理方式的效果*

這個選擇在單張照片上不痛不癢，但在三種情況下會造成問題：拼接時(第 13 章)接縫處會出現亮度不連續;批次分塊處理大圖時(第 17 章 AOI 大圖分割檢測)每塊的邊緣都有瑕疵;以及第 6 章的頻率域處理——DFT 隱含「影像週期性延拓」的假設，等同於 BORDER_WRAP，這會在頻譜上產生十字狀的假亮線。理解邊界處理，能幫你診斷許多莫名其妙的邊緣瑕疵。

### 5.3.3 可分離濾波與運算量

若一個 m×m 的核可以拆成一個垂直向量與一個水平向量的外積，就稱為可分離(Separable)。此時先做一次水平一維卷積、再做一次垂直一維卷積，結果與二維卷積相同，但每個像素的乘法次數從 $m^2$ 降為 2m。

以 15×15 的高斯核為例：225 次乘法降為 30 次，快了七倍以上;若是 51×51，則從 2601 降為 102，快了二十五倍。高斯核與均值核都是可分離的，OpenCV 的 GaussianBlur 內部就這麼做。

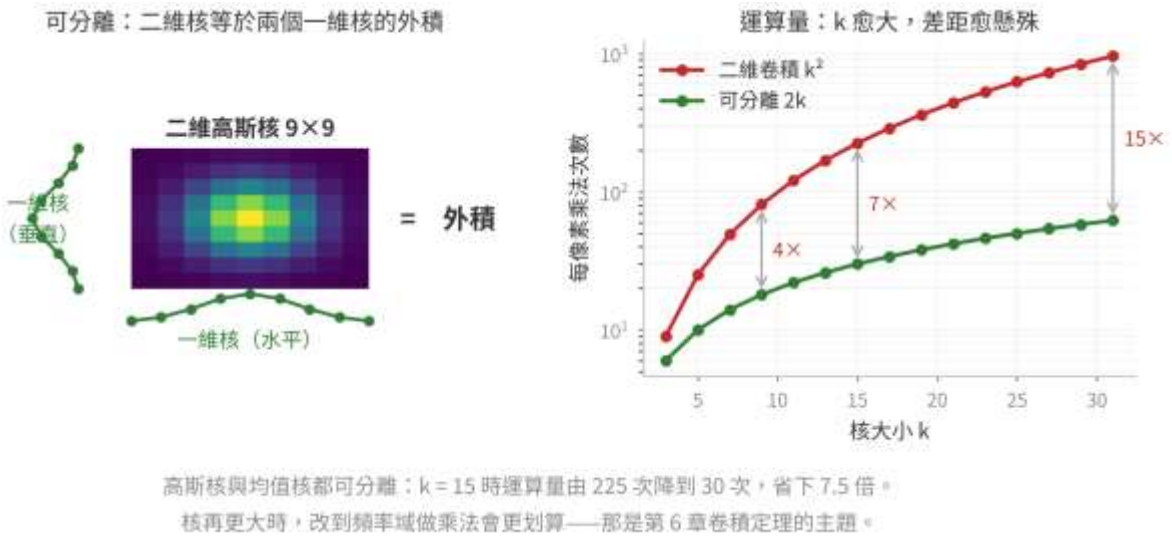



*圖 5-10　可分離濾波：原理與運算量*

這個技巧是即時影像處理能在 Jetson 或手機上跑得動的關鍵之一，也預告了第 6.5.2 節的工程判準：當核大到連可分離都不划算時，就該轉到頻率域了。三者(空間域直接卷積、可分離卷積、頻率域相乘)構成一個依核大小選擇的完整光譜。

### 5.3.4 推導：高斯核為什麼可分離，中值為什麼不行

可分離不是巧合，而是高斯函數的一個代數性質。二維高斯核為 $G(x, y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2 + y^2) / 2\sigma^2)$。由於指數相加等於指數相乘，可以直接拆開：$\exp(-(x^2 + y^2) / 2\sigma^2) = \exp(-x^2 / 2\sigma^2) \cdot \exp(-y^2 / 2\sigma^2)$，因此 $G(x, y) = g(x) \cdot g(y)$，其中 g 是一維高斯。二維核恰好是兩個一維核的外積。

有了這個分解，卷積就能拆成兩次一維運算：$(f * G)(x, y) = \Sigma_v g(v) \cdot [\Sigma_u f(x - u, y - v) \cdot g(u)]$。括號內是一次水平方向的一維卷積，括號外是一次垂直方向的一維卷積。原本每個像素要做 $k^2$ 次乘法，現在只要 $k + k = 2k$ 次。本書以 9×9 的高斯核實測，二維卷積與可分離實作的輸出最大差異為 $5.6 \times 10^{-16}$，純粹是浮點誤差——兩者在數學上完全等價。均值核同樣可分離，因為所有權重都是 $1/k^2 = (1/k) \times (1/k)$，也是一個外積。

中值濾波則不可分離，而且這不是「還沒有人想到好方法」，而是原理上就不行。中值是一個排序統計量，先取各列的中值再取這些中值的中值，一般並不等於整個鄰域的中值。舉一個 3×3 的反例：[[1,5,9],[8,8,0],[0,2,0]]，九個數排序後的中值是 2;但三列的中值分別為 5、8、0，再取中值得到 5——答案不同。這就是為什麼 cv2.medianBlur 在核變大時速度掉得特別快，也是實務上中值濾波很少使用大核的原因。

## 5.4 平滑濾波：對付雜訊

1. 均值濾波(Box Filter)：鄰域取算術平均。最快最簡單，但邊緣與細節同樣被抹平，且方形核容易產生方向性偽影(從第 6 章的觀點看，它的頻率響應是有振盪旁瓣的 sinc 型低通，這正是偽影的來源)。
2. 高斯濾波(Gaussian Blur)：以高斯函數加權，中心權重最大、離得愈遠愈小。平滑效果自然，是最常用的平滑濾波;參數為核大小與標準差 sigma,sigma 愈大

愈模糊。它同時是第 9 章 Canny 邊緣偵測的第一步、第 11 章小波多尺度與第 13 章 SIFT 尺度空間的基礎——高斯是整本書出現最頻繁的濾波器。

3. 中值濾波(Median Blur)：取鄰域的中位數而非平均。對椒鹽雜訊(隨機黑白點)有奇效——因為極端值不會影響中位數;且比均值更能保留邊緣銳利度。缺點是運算較慢，且大核心會讓影像產生塊狀感。它是非線性運算，因此沒有對應的頻率響應(第 6.6.2 節會說明這個限制的意義)。

4. 雙邊濾波(Bilateral Filter)：同時考慮「空間距離」與「像素值差異」兩個權重，只平均那些「離得近且顏色也相近」的鄰居，因此能在去雜訊的同時保住邊緣。代價是運算量大;它正是手機美顏「磨皮但不糊掉五官」的原理，也是第 13 章車牌偵測前處理的常用選擇。

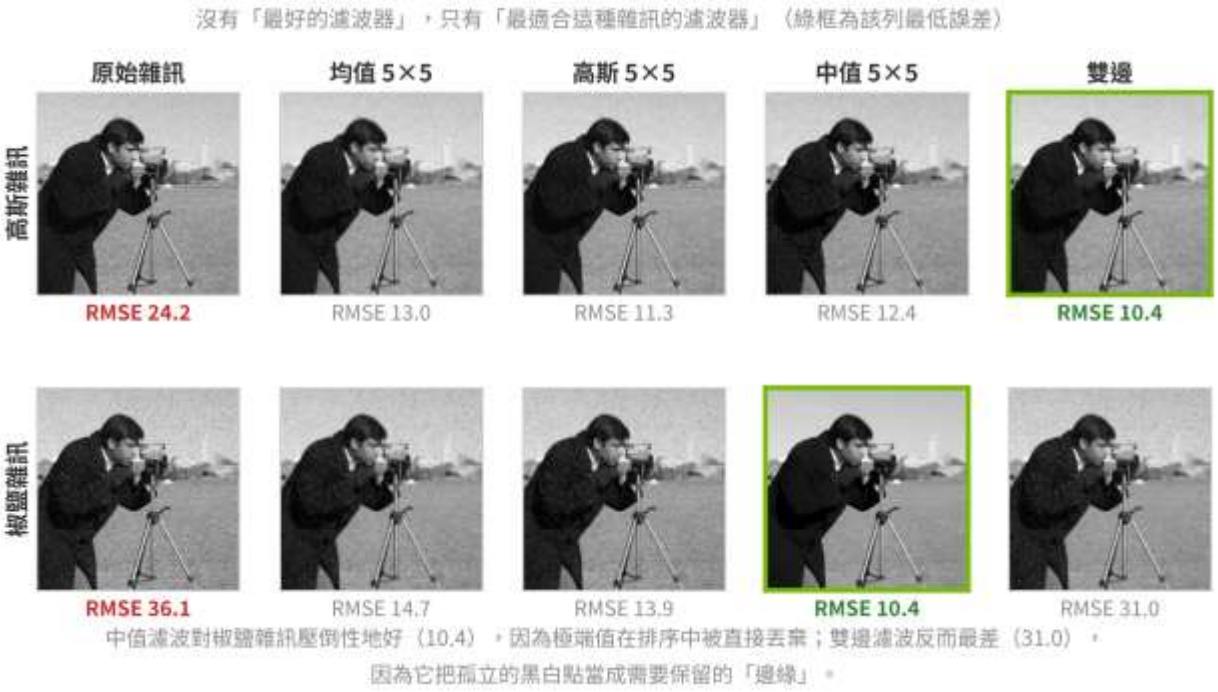



*圖 5-11　四種平滑濾波對兩種雜訊的表現*

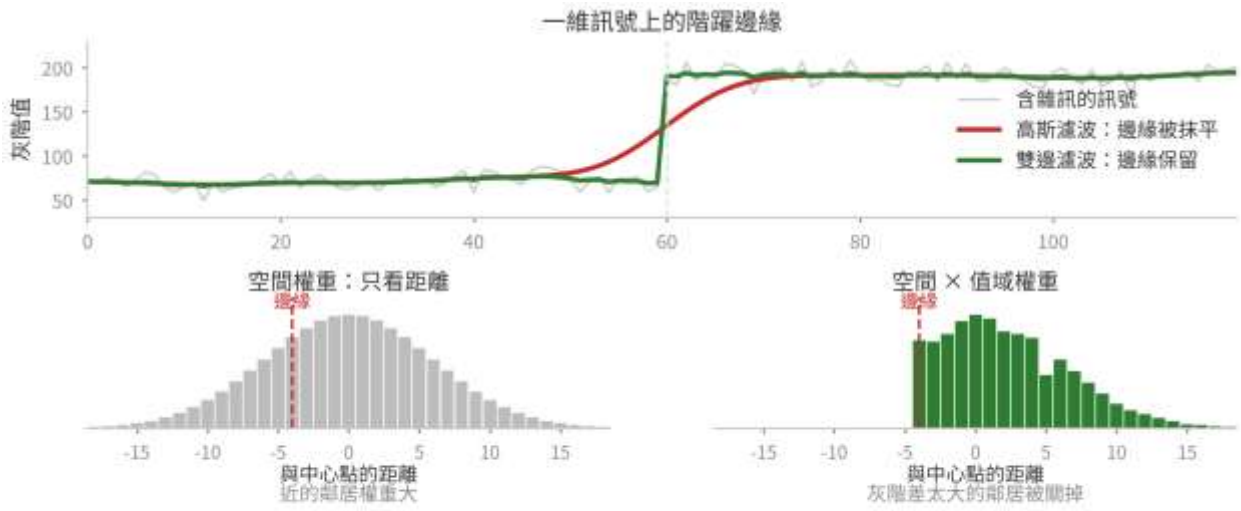



*圖 5-12　雙邊濾波為什麼能保留邊緣*

| 濾波器 | 擅長的雜訊 | 邊緣保留 | 速度 | OpenCV 函式 |
|---|---|---|---|---|
| 均值 | 高斯雜訊(普通) | 差 | 最快 | cv2.blur |
| 高斯 | 高斯雜訊(佳) | 差 | 快 | cv2.GaussianBlur |
| 中值 | 椒鹽雜訊(極佳) | 中等 | 中等 | cv2.medianBlur |
| 雙邊 | 高斯雜訊(佳) | 優異 | 慢 | cv2.bilateralFilter |

*表 5-1 四種平滑濾波比較*

**程式 5-3 四種平滑濾波與效能比較**

```
import cv2, numpy as np

img = cv2.imread("noisy.jpg")

mean   = cv2.blur(img, (5, 5))
gauss  = cv2.GaussianBlur(img, (5, 5), sigmaX=1.5)
median = cv2.medianBlur(img, 5)              # 核大小須為奇數
bil    = cv2.bilateralFilter(img, d=9,
                             sigmaColor=75, sigmaSpace=75)

# 實驗建議:在原圖上分別加高斯雜訊與椒鹽雜訊,
# 用第 3 章的 cv2.PSNR 量化四種濾波的還原成效
clean = cv2.imread("clean.jpg")
for name, out in [("均值", mean), ("高斯", gauss),
                  ("中值", median), ("雙邊", bil)]:
    print(f"{name}: PSNR = {cv2.PSNR(clean, out):.2f} dB")
```

```
# 驗證可分離濾波的效能(5.3.3 節)
import time
k = 31
t0 = time.time(); cv2.GaussianBlur(img, (k, k), 0)
print(f"高斯(可分離){time.time()-t0:.4f} 秒")
kernel = np.ones((k, k), np.float32) / (k * k)
t0 = time.time(); cv2.filter2D(img, -1, kernel)
print(f"filter2D(二維){time.time()-t0:.4f} 秒")
```

## 5.5 銳化濾波：強調細節

平滑靠平均，銳化則靠微分：影像中的邊緣就是灰階劇烈變化之處，而微分對這種變化最敏感。這也預告了第 9 章——邊緣偵測用的是同一套數學，只是目的從「強調邊緣」變成「找出邊緣的位置」。

### 5.5.1 Laplacian 銳化

拉普拉斯運算子(Laplacian)是二階微分的離散近似，常見核為中心 -4(或 -8)、四鄰為 1 的 3×3 矩陣。單獨使用 Laplacian 得到的是「邊緣圖」，把它加回原圖才是銳化：$g = f - c\cdot\nabla^2 f$。




*圖 5-13　Laplacian 銳化的四個步驟*

二階微分的特性是各向同性(不分方向)、定位精準，但對雜訊極度敏感——因為微分兩次會把雜訊放大得更兇。這正是第 9.3.2 節 LoG(高斯拉普拉斯)要先做高斯平滑的原因，也是本節結尾「先去雜訊再銳化」原則的根據。

## 5.5.2 Unsharp Masking 與 High-Boost

更常用的路線是遮罩銳化(Unsharp Masking)，名字有點反直覺——它其實來自傳統暗房技術：先把影像模糊化，用原圖減去模糊版得到「細節層」(即被模糊抹掉的高頻)，再把細節層加權加回原圖。公式為 g = f + k·(f - blur(f))。

當 k = 1 稱為 Unsharp Masking,k > 1 則稱為 High-Boost Filtering，能同時提亮並強化細節。從第 6 章的頻率觀點看，這正是「高頻強調濾波」——本章與第 6 章做的是同一件事，只是一個用空間域的減法實現、一個用頻率域的加權實現。這種「同一操作的兩張面孔」是理解影像處理的關鍵心法。




*圖 5-14　遮罩銳化的原理與 k 值的影響*

## 5.5.3 銳化的代價

銳化有三個必須知道的代價。第一，過衝(overshoot):k 太大時邊緣兩側會出現白邊與黑邊，俗稱光暈，看起來不自然——這與第 6 章的振鈴效應同源。第二，雜訊放大：高頻成分中雜訊佔了很大比重，銳化會一併放大它們。第三，無中生有的錯覺：銳化並沒有把模糊掉的資訊找回來，它只是把邊緣兩側的對比拉大，製造出「更清楚」的視覺印象——真正的還原是第 7 章的反卷積。

因此有一條實務鐵則：銳化之前要先適度去雜訊，順序不能顛倒。若先銳化再去雜訊，雜訊已經被放大，再平滑只會連同細節一起抹掉。這個「順序決定成敗」的原則，在第 9 章的 Canny(先平滑再求梯度)中會再次出現。

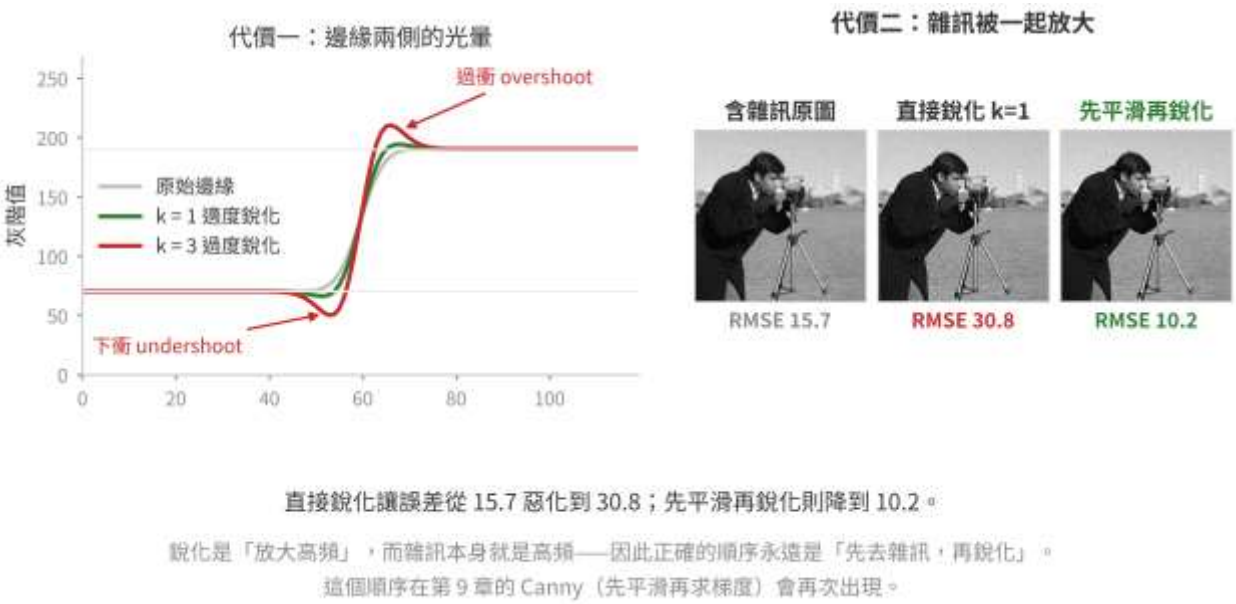



*圖 5-15　銳化的兩個代價：光暈與雜訊放大*

**程式 5-4 銳化與正確的處理順序**

```
import cv2, numpy as np

img = cv2.imread("soft.jpg")

# --- 方法一:Laplacian 銳化 ---
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
sharp1 = cv2.convertScaleAbs(img - 0.5 * lap)

# --- 方法二:Unsharp Masking / High-Boost(實務首選) ---
blur = cv2.GaussianBlur(img, (0, 0), sigmaX=3)
k = 1.5                                    # k>1 即 High-Boost
sharp2 = cv2.addWeighted(img, 1 + k, blur, -k, 0)

# --- 正確順序:先去雜訊,再銳化 ---
denoised = cv2.bilateralFilter(img, 9, 50, 50)   # 保邊去雜訊
blur2 = cv2.GaussianBlur(denoised, (0, 0), 3)
good = cv2.addWeighted(denoised, 2.0, blur2, -1.0, 0)

# --- 錯誤順序:先銳化再去雜訊(雜訊已被放大) ---
bad = cv2.bilateralFilter(sharp2, 9, 50, 50)

cv2.imwrite("sharp_correct.jpg", good)
cv2.imwrite("sharp_wrong.jpg", bad)
# 並排比較:錯誤順序的結果細節較糊、雜點殘留較多
```

### 5.5.4 推導：遮罩銳化其實是一個高通濾波

遮罩銳化的公式 $g = f + k \cdot (f - f * G)$ 看起來像是「兩個步驟」，但把它整理一下會發現，它其實只是「一次卷積」，而且是一個我們到第 6 章才會正式命名的東西。

把式子展開：$g = (1 + k) \cdot f - k \cdot (f * G)$。由於 f 本身可以寫成 $f * \delta$（$\delta$ 是單位脈衝，即中心為 1、其餘為 0 的核），整個式子可以合併成 $g = f * [(1 + k) \cdot \delta - k \cdot G]$。也就是說，遮罩銳化等於用單一個核 $(1 + k)\delta - kG$ 做一次卷積。這個核的權重總和為 $(1 + k) - k = 1$（因為高斯核的總和為 1），所以影像的整體亮度不變——這與 5.3 節說的「平滑核總和為 1」是同一個道理。

更有意思的是它在頻率域的樣子（第 6 章會給出完整工具）。這個核的轉移函數為 $H = (1 + k) - k \cdot \hat{G}$，而高斯是典型的低通濾波：在零頻處 $\hat{G} = 1$，頻率愈高 $\hat{G}$ 愈趨近 0。代入可得：零頻處 $H = (1 + k) - k = 1$，高頻處 $H \to (1 + k)$。本書以 $\sigma = 3$ 的高斯核實測，$k = 1$ 時 H 由 1.0000 平滑地升到 2.0000，$k = 2$ 時升到 3.0000，與理論完全吻合。

這解釋了三件事。第一，為什麼它叫 High-Boost——低頻原封不動通過（增益為 1），高頻被放大到 $(1 + k)$ 倍。第二，為什麼雜訊會被放大：雜訊的能量集中在高頻，正好落在增益最大的區段。第三，為什麼第 6 章要花整章談頻率域——空間域裡看似兩個獨立步驟的操作，到了頻率域只是一條增益曲線，設計與分析都變得直觀得多。

## 5.6 多影格合成：以軟體突破硬體限制

本節把叫獸開講的核心技術展開，它也是計算攝影(Computational Photography)這個領域的縮影。

### 5.6.1 為什麼平均能去雜訊

假設雜訊是隨機且零均值的(高斯雜訊符合此假設)，那麼對同一場景拍 N 張、逐

像素平均後：訊號部分因為每張都相同，平均後不變;雜訊部分因為隨機且正負相消，其標準差降為原來的 N 平方根分之一。平均 4 張，雜訊減半;平均 16 張，雜訊剩四分之一;平均 100 張，剩十分之一。

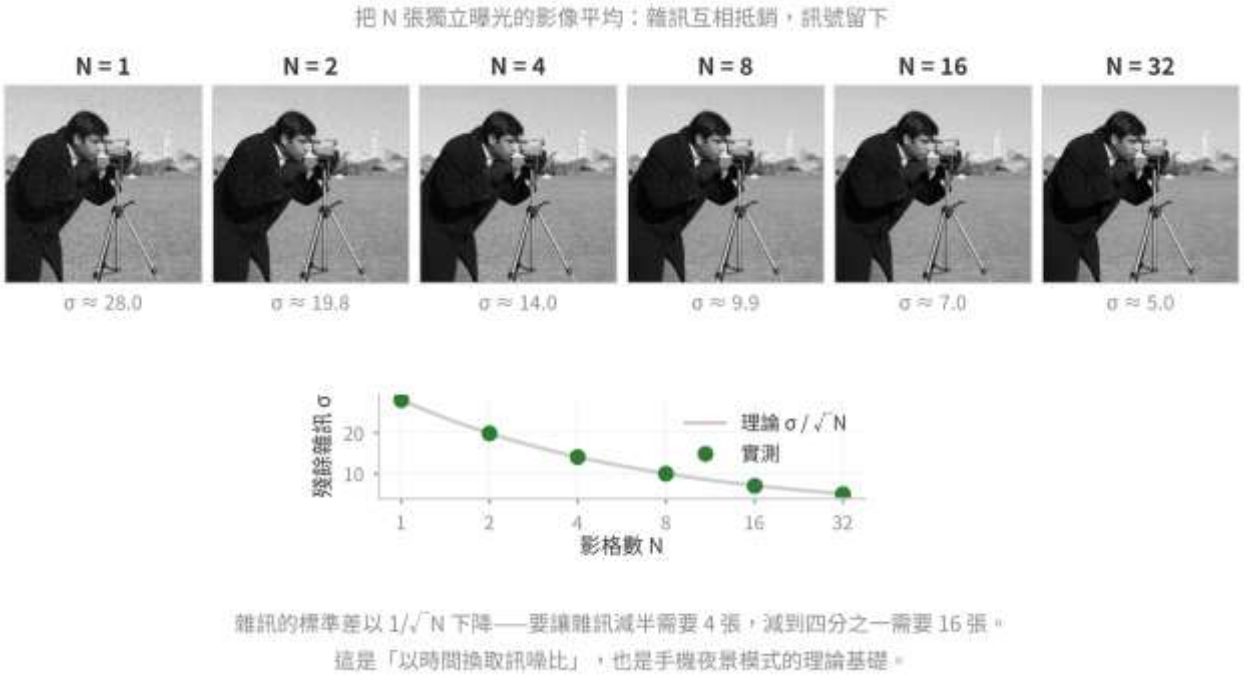



*圖 5-16　多影格平均：雜訊隨 1/√N 下降*

這正是叫獸開講思考題的答案，也解釋了「訊噪比隨曝光時間平方根上升」的物理直覺——更長的曝光就是收集更多光子，而更多的獨立樣本平均起來，隨機波動自然變小。天文攝影界稱這個技術為「曝光堆疊」，已使用數十年。

## 5.6.2 多影格平均與空間平均的關鍵差異

這裡有一個極重要的對照，值得細細體會：5.4 節的空間平滑，平均的是「同一時間的不同像素」——但鄰近像素的訊號未必相同(邊緣兩側就完全不同)，因此平均會模糊邊緣。而多影格平均，平均的是「不同時間的同一個像素」——訊號理論上完全相同，只有雜訊在變。

結論是：多影格平均能在幾乎不損失任何細節的前提下降低雜訊，這是單一影格去雜訊永遠做不到的。這也是為什麼所有嚴肅的低光攝影、天文攝影、以及第 7 章的醫學影像，都盡可能採用多影格方案。

### 5.6.3 對齊：多影格方案的真正難點

但前提是「同一個像素」——手持拍攝必然有位移與旋轉，若不對齊就直接平均，結果是一張模糊的照片。因此對齊才是多影格合成的真正技術核心，也是 Night Sight 所謂 robust merge 的重點。




*圖 5-17　對齊是多影格合成的真正難點*

對齊的方法本書後續都會學到：第 6.7.2 節的相位相關法(利用傅立葉的平移性質，快速且對亮度變化免疫)、第 13 章的特徵匹配加 RANSAC(能處理旋轉與透視變化)、以及第 12 章的光流。而 Night Sight 更進一步採用分塊對齊，並丟棄運動過大的區塊——因為場景中可能有移動的人或車，強行對齊會產生鬼影。

這條技術鏈完整地展示了本書各章如何協作：第 5 章提供合成的原理、第 6 或 13 章提供對齊的工具、第 4 章提供轉換的數學、第 3 章提供品質的評估。一個看似單純的「夜景模式」，背後是整本書的知識。

**程式 5-5 窮人版夜景模式**

```
import cv2, numpy as np, glob

# --- 窮人版夜景模式:多影格平均 ---
files = sorted(glob.glob("burst/*.jpg"))      # 連拍的一疊暗景
ref = cv2.imread(files[0])
acc = np.zeros_like(ref, dtype=np.float32)
n = 0
```

```
ref_gray = cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY).astype(np.float32)
win = cv2.createHanningWindow(ref_gray.shape[::-1], cv2.CV_32F)

for f in files:
    img = cv2.imread(f)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)

    # 對齊:相位相關法求位移(第 6 章)
    (dx, dy), _ = cv2.phaseCorrelate(ref_gray, gray, win)
    M = np.float32([[1, 0, -dx], [0, 1, -dy]])       # 第 4 章仿射
    aligned = cv2.warpAffine(img, M, img.shape[1::-1])

    acc += aligned.astype(np.float32)
    n += 1

merged = (acc / n).astype(np.uint8)

# 後處理:提亮 + 局部對比 + 補回銳利度
merged = gamma_correct(merged, 2.0)                  # 5.1 節
lab = cv2.cvtColor(merged, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
l = cv2.createCLAHE(2.0, (8, 8)).apply(l)            # 5.2 節
merged = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)

blur = cv2.GaussianBlur(merged, (0, 0), 2)
merged = cv2.addWeighted(merged, 1.5, blur, -0.5, 0)  # 5.5 節
cv2.imwrite("night_sight_poor_man.jpg", merged)

print(f"合併 {n} 張,理論雜訊降為 1/{np.sqrt(n):.1f}")
```

### 5.6.4 推導：1/√N 從哪裡來，又在什麼時候失效

多影格平均的 1/√N 法則值得完整推導一次，因為它的三個前提，每一個在實務上都會被打破，而知道是哪一個被打破，才知道該怎麼補救。

設第 i 張影格的某個像素為 $f_i = s + n_i$，其中 $s$ 是不變的訊號、$n_i$ 是雜訊。假設 $n_i$ 為零均值（$E[n] = 0$）、變異數相同（$\mathrm{Var}[n] = \sigma^2$）、且各影格之間互相獨立。$N$ 張平均後的值為 $\frac{1}{N}\sum f_i = s + \frac{1}{N}\sum n_i$。訊號 $s$ 原封不動保留;雜訊項的變異數為 $\mathrm{Var}\left[\frac{1}{N}\sum n_i\right] = \frac{1}{N^2}\cdot\sum \mathrm{Var}[n_i] = \frac{1}{N^2}\cdot N\sigma^2 = \sigma^2 / N$。開根號後，標準差為 $\sigma / \sqrt{N}$。這就是 1/√N 的來源——要讓雜訊減半需要四張，減到四分之一需要十六張。

三個前提各自對應一種實務上的失效模式。其一，零均值：感測器的暗電流會在每張影格加上同一個正偏移，它不是零均值，平均再多張也不會消失——解法是先拍

一張蓋上鏡頭的暗影格再相減。其二，互相獨立：固定圖樣雜訊（每個像素的增益與偏移差異）在每張影格中完全相同，平均不但無法消除，反而會因為隨機雜訊下降而相對變得更明顯——解法是平場校正。其三，訊號不變：手持拍攝會讓場景在影格間位移，此時「同一個像素」對應到的已經不是同一處景物，平均就會順帶模糊訊號——解法是對齊，也就是 5.6.3 節的主題。

把這三件事放在一起看，就能理解為什麼一套可用的夜景模式遠不只是「相加除以 N」：它必須先扣掉不隨機的部分（暗電流、固定圖樣），再對齊會動的部分（手震、物體移動），最後才輪到平均。理論給出的 1/√N 是上限，工程的任務是盡量逼近它。

## 5.7 應用案例

### 5.7.1 案例一：工業影像前處理管線

情境：AOI 檢測系統要在打光不均的工件表面找出刮痕。若直接做第 9 章的門檻分割，亮區的背景會被誤判為前景、暗區的瑕疵則被漏掉。

標準管線：轉灰階 → 用 CLAHE 拉平局部對比(5.2.3 節)→ 用雙邊濾波或中值濾波去除雜點但保住瑕疵邊緣(5.4 節)→ 適度銳化強化瑕疵的邊緣對比(5.5 節)→ 交給第 9 章分割。

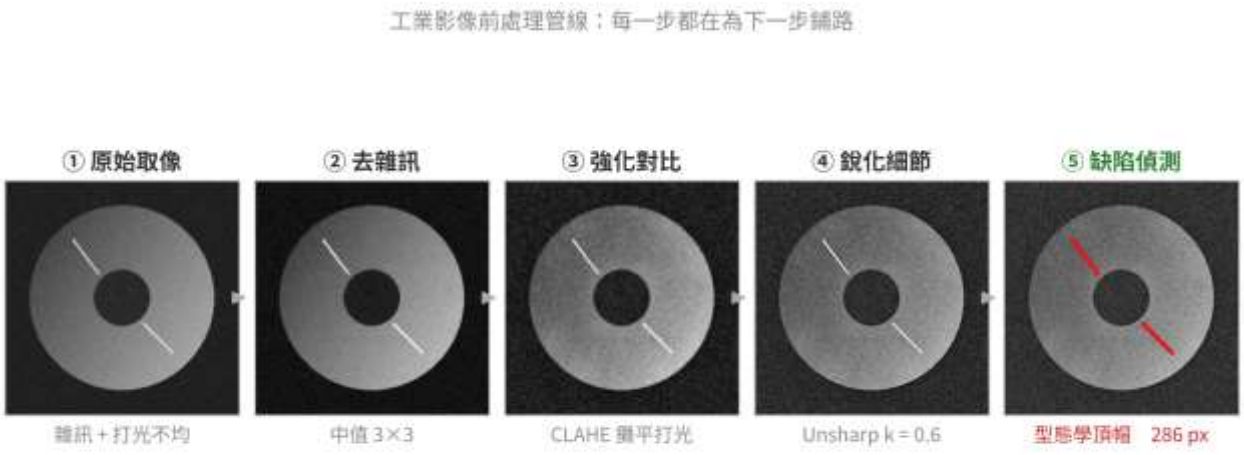



*圖 5-18　工業影像前處理管線的五個步驟*

這個案例的關鍵教訓是：前處理的目標不是「讓影像好看」，而是「讓後續演算法好做」。有時候一張經過強化的影像在人眼看來反而怪異(對比過強、色彩失真)，但只要它讓分割與量測更準確，就是成功的前處理。這也是第 5 章與第 7 章在判準上的又一個差異——強化的成敗，取決於下游任務的表現。

### 5.7.2 案例二：醫學影像的視窗化與強化

情境：承第 3.7.2 節，CT 影像以 12 位元儲存(4096 階)，但螢幕只能顯示 256 階。醫師需要依觀察目標選擇「視窗」：看肺部用一組視窗中心與寬度、看骨骼用另一組、看軟組織再一組。

視窗化本質上就是本章的灰階轉換——把 12 位元中的某一段線性拉伸到完整的顯示範圍，超出範圍的一律設為全黑或全白。這說明了一個重要觀念：同一份資料，不同的轉換會揭示不同的資訊。這也是為什麼高位元深度的原始資料必須完整保存——它承載了顯示時尚未取用的資訊。

醫學應用也有其特殊的謹慎：過度的強化(尤其是銳化)可能製造出不存在的邊緣、或讓微小病灶的邊界看起來比實際清楚，誤導判讀。這與第 7 章「還原 vs. 腦補」、第 16 章的技術倫理是同一個主題：在高風險領域，「看起來更清楚」與「更接近真實」

必須嚴格區分。

### 5.7.3 案例三：計算攝影與手機夜景

情境：承叫獸開講，現代手機的相機模組在物理上很受限——鏡頭小、感光元件小、進光量少。但拍出來的照片卻常常勝過某些專業相機的單張直出，原因就在計算攝影：用軟體與運算補上硬體的不足。

這個案例最能說明本章的價值定位。回顧全書的三種思路：硬體不足時，可以換更好的硬體(貴)、可以用數學還原(第 7 章，但受物理上限所限)、也可以「多拍幾張再合成」(本章 5.6 節)。第三條路往往最務實——它把「空間上的限制」轉換成「時間上的成本」，而時間通常比硬體便宜。這個「換一種資源來解決問題」的思維，值得學生反覆體會。

### 5.7.4 案例四：訓練資料的前處理與正規化

情境：承第 14 至 16 章，訓練深度學習模型前，所有影像都要經過統一的前處理。這裡本章的技術扮演兩個角色。

其一是正規化：把像素值從 0 到 255 縮放到 0 到 1 或標準化為零均值單位變異數。這在數學上是最單純的點運算(本章 5.1 節)，卻對訓練的收斂速度有顯著影響——因為神經網路的梯度下降對輸入尺度相當敏感。

其二是條件統一：若訓練資料來自不同的光照條件(有些在晴天拍、有些在陰天拍)，用 CLAHE 或白平衡統一它們，能減少模型需要學習的變異、提升泛化能力。但這裡有個微妙的取捨——若推論時的影像也會有同樣的光照變化，那麼「保留變化並用資料增強讓模型適應」可能比「強行統一」更好。這正是第 7.7.3 節與第 15 章討論過的「還原 vs. 資料增強」抉擇，在前處理階段的體現。

## 5.8 本章與全書的觀念連結總表

| 本章觀念 | 後續應用的章節 | 如何延伸 |
|---|---|---|
| 對數轉換 | 第 6 章 | 顯示傅立葉頻譜前的動態範圍壓縮 |
| Gamma 與線性空間 | 第 3、7、8 章 | RAW 為線性;還原與白平衡須留意非線性 |
| 直方圖 | 第 7、9 章 | 雜訊型態診斷;Otsu 自動門檻的基礎 |
| CLAHE | 第 9、10、16 章 | 分割前處理;AOI 打光不均;訓練資料統一 |
| 卷積與濾波核 | 第 6、15 章 | 卷積定理;CNN 的卷積層即可學習的核 |
| 高斯濾波 | 第 9、11、13 章 | Canny 首步;小波多尺度;SIFT 尺度空間 |
| 中值與雙邊濾波 | 第 6、7、13 章 | 非線性故無頻率響應;去雜訊工具箱;車牌前處理 |
| 可分離濾波 | 第 6、17 章 | 運算量取捨;邊緣裝置的即時性 |
| Unsharp / High-Boost | 第 6、7 章 | 等同頻率域高頻強調;與反卷積的根本差異 |
| 多影格合成 | 第 6、12、13 章 | 相位相關與特徵匹配對齊;視訊的時間維度 |

*表 5-2 第 5 章與全書其他章節的觀念連結*

## 程式實作 Lab 5

### Lab 5-1 灰階轉換與直方圖診斷

目標：建立「先診斷、再開藥」的習慣。(a)準備三張影像：曝光不足、過曝、對比不足，各自繪製直方圖並描述其形狀特徵;(b)對每張套用適當的處理(Gamma、對數、等化)，說明你的選擇理由;(c)實作程式 5-1 的 LUT 版 Gamma 校正，與「逐像素用 np.power 計算」比較執行時間(呼應第 2 章向量化);(d)驗證 Gamma 編碼的非線性：對一張 RAW(或以 γ=1/2.2 模擬的線性影像)與其 JPEG 版本，比較「兩個區域的亮度

比值」是否相同;(e)結論：寫出一張決策流程圖——看到什麼樣的直方圖，該用哪種轉換?

### Lab 5-2 等化、CLAHE 與彩色陷阱

目標：掌握對比強化的正確做法。(a)手算驗證 5.2.2 節的範例(L=8、64 像素)，再用程式驗證你的答案;(b)對一張「同時有明亮窗戶與陰暗室內」的影像，比較全域等化與 CLAHE 的結果，說明前者為何顧此失彼;(c)調整 CLAHE 的 clipLimit(1、2、4、10)與 tileGridSize(4×4、8×8、16×16)，觀察各自的影響並找出最佳組合;(d)彩色陷阱實驗：對同一張彩色影像，分別用「BGR 三通道各自等化」與「轉 LAB 只等化 L 通道」，並排比較色偏程度;(e)觀察等化在平坦區域放大雜訊的現象，並思考如何緩解。

### Lab 5-3 濾波器擂台

目標：為第 7 章的去雜訊工具箱打底。(a)對一張乾淨影像分別加入高斯雜訊與椒鹽雜訊;(b)用均值、高斯、中值、雙邊四種濾波各處理一次，製作 2×4 的 PSNR 與 SSIM 比較表(用第 3 章 Lab 3-3 建立的評估函式);(c)寫出結論：哪種雜訊該配哪種濾波器?為什麼中值對椒鹽近乎完美、對高斯卻無特別優勢?(d)實測可分離濾波：比較 GaussianBlur 與同尺寸 filter2D 的執行時間，核大小從 3 到 51，繪製曲線;(e)銳化順序實驗：比較「先去雜訊再銳化」與「先銳化再去雜訊」的結果，以 PSNR 與目視雙重驗證。

### Lab 5-4 窮人版夜景模式

目標：重現叫獸開講的核心技術。(a)用手機或 USB 相機以固定姿勢連拍同一暗景 16 張;(b)先不對齊直接平均，計算雜訊標準差並驗證思考題：平均 16 張後是否約降為四分之一?(c)手持連拍(必然有晃動)，比較「直接平均」與「先用相位相關對齊再平均」的清晰度差異;(d)以程式 5-5 完成完整管線：對齊平均 → Gamma 提亮 → CLAHE 局部對比 → Unsharp 補銳度;(e)以第 3 章的 PSNR/SSIM 比較「單張」、「多張平均」、「完整管線」三者相對於長曝光參考照的差距;(f)進階挑戰：在畫面中放一個移動的物

體，觀察直接平均會產生什麼鬼影，並思考 Night Sight 的「分塊對齊並丟棄運動區塊」如何解決它。

### Lab 5-5 邊界、可分離與效能實測

目標：把 5.3 節的三個工程細節實際量出來。(a)對同一張影像以 31×31 的高斯核模糊，分別用 constant、replicate、reflect、reflect_101、wrap 五種邊界處理，把影像四個角落放大並列(重現圖 5-9)，說明哪一種在你的應用上最合適;(b)計算這張影像有多少比例的像素受邊界處理影響，與理論值比對;(c)以 cv2.filter2D 與 cv2.sepFilter2D 實作同一個均值濾波，對 k = 3 到 31 各量測十次取平均，畫出時間對 k 的曲線;(d)你會發現 k 很大時 filter2D 的時間不再成長——查閱文件說明原因，並連結到第 6 章的卷積定理;(e)驗證可分離性：以 cv2.getGaussianKernel 取出一維核，計算其外積，與 cv2.getGaussianKernel 的二維版本比較是否相同。

### Lab 5-6 建立你自己的前處理管線

目標：把 5.7.1 節與圖 5-18 的管線套到你自己拍的影像上，並且用數字證明每一步都有幫助。(a)拍攝一組有「打光不均」與「可見雜訊」的照片(刻意用側光、調高 ISO 或在昏暗處拍攝)，物件可以是零件、硬幣或電路板，並刻意製造一處可見的瑕疵;(b)依「去雜訊 → 強化對比 → 銳化」的順序建立管線，每一步都存下中間結果;(c)為每一步定義一個量化指標——例如平坦區的標準差(衡量雜訊)、瑕疵區與周圍背景的灰階差(衡量對比)，逐步記錄數值變化;(d)刻意把順序顛倒(先銳化再去雜訊)，比較指標與目視結果，驗證 5.5.3 節的論點;(e)把最後的結果交給第 10 章的型態學頂帽運算找出瑕疵，記錄偵出面積與良品的誤報面積;(f)寫下一段三百字的檢討：哪一步的收益最大?哪一步其實可以省略?

## 本章重點整理

- 空間域處理框架 g = T[f];鄰域為單一像素時稱點運算(灰階轉換)，擴大鄰域即空間濾波。本章屬影像強化(主觀、讓影像好看或好用)，與第 7 章的還原(客觀、逼近原始)判準不同。
- 灰階轉換：負片(醫學影像)、對數(壓縮動態範圍、顯示頻譜)、Gamma 冪次法則(γ>1 提亮暗部);螢幕 γ=2.2 的歷史使 JPEG 非線性，精確光度計算須先轉回線性;以 LUT 查表實作最有效率。
- 直方圖是診斷工具，也是第 7 章雜訊辨識與第 9 章 Otsu 門檻的基礎;等化以累積分布函數為映射，但會放大雜訊且不可控;CLAHE 分格局部等化並以 clipLimit 削頂抑制雜訊，實務上優於全域等化;彩色影像應只等化亮度通道。
- 卷積需將核旋轉 180 度，相關則否(OpenCV filter2D 實為相關);CNN 的卷積層即可學習的濾波核。邊界處理影響拼接、分塊與頻域結果;可分離濾波將 $m^2$ 次乘法降為 2m 次，是即時處理的關鍵。
- 平滑濾波：均值(快但有偽影)、高斯(自然，全書最常用)、中值(專治椒鹽、非線性)、雙邊(保邊緣、磨皮原理)。
- 銳化基於微分：Laplacian 為二階微分(對雜訊極敏感);Unsharp Masking 以原圖減模糊得細節層再加回，k>1 即 High-Boost，等同頻率域的高頻強調。銳化會過衝與放大雜訊，且未真正找回資訊，必須先去雜訊再銳化。
- 多影格平均使雜訊標準差降為 N 平方根分之一;相較空間平均(同時刻不同像素，會模糊邊緣)，多影格平均(不同時刻同一像素)幾乎不損失細節，但需先對齊(第 6、12、13 章的技術)。

## 習題

1. 點運算與鄰域運算的差別是什麼?各舉兩個例子。本章的「強化」與第 7 章的「還原」在判準上有何根本不同?

   *提示與參考解答：點運算的輸出只取決於該像素自己的值(如負片、Gamma 校正、直方圖等化);鄰域運算則取決於周圍一片像素(如高斯模糊、中值濾波、銳化)。強化*

*追求主觀的「看起來更好或更適合後續處理」，如 Night Sight 刻意壓黑陰影以營造夜的氛圍;還原則是客觀的逆問題，先建立劣化模型再反推最接近原始的估計，可用 PSNR、SSIM 量化驗收。一句話：強化是化妝，還原是手術。*

2. Gamma 校正中，γ = 2.2 與 γ = 0.45 分別使影像變亮或變暗?為什麼 JPEG 的像素值不是線性正比於光強度?這在什麼情況下會造成問題?

   *提示與參考解答：公式 s = c·r^γ 中，實作上常以 1/γ 為指數(如程式 5-1);依課文定義，γ>1 提亮暗部、γ<1 壓暗。JPEG 非線性的原因：早期映像管螢幕的亮度響應近似 γ=2.2 的冪次曲線，影像儲存時預先做了反向的 Gamma 編碼來抵消。問題發生在精確光度量測時——直接用 JPEG 像素值計算「亮度比值」會得到錯誤答案，須先反 Gamma 轉回線性空間(第 7 章還原、第 8 章白平衡皆須留意)。*

3. 為什麼顯示傅立葉頻譜前通常要取對數?這利用了對數轉換的什麼特性?

   *提示與參考解答：因為頻譜的直流分量(整張影像的平均亮度)數值往往比其他成分大好幾個數量級，不取對數的話整張頻譜圖只看得到中央一個亮點、其餘全黑。對數轉換大幅拉伸暗部、壓縮亮部，能壓縮極大的動態範圍，讓微弱的高頻成分也看得見。這是對數轉換「適合處理動態範圍極大影像」的最佳實例。*

4. 某 L = 8 的影像共 64 像素，各灰階像素數為 8、10、16、14、8、5、2、1。請計算直方圖等化後的映射表，並說明會發生什麼細節損失。

   *提示與參考解答：累積機率依序為 0.125、0.281、0.531、0.750、0.875、0.953、0.984、1.000;乘以 7(L-1)並四捨五入得映射：0→1、1→2、2→4、3→5、4→6、5→7、6→7、7→7。細節損失：原本不同的灰階 5、6、7 都映射到 7，合併成同一階，這三階之間的差異永久消失。這說明等化雖擴展了整體對比，卻可能犧牲高灰階區的細節。*

5. 說明 CLAHE 相較全域等化的兩項改良，並解釋 clipLimit 的作用與名稱由來。

   *提示與參考解答：改良一：分格局部等化(如 8×8)，各格依自己的統計量調整，再以雙線性內插消除接縫，解決「同時有明亮窗戶與陰暗室內時全域等化顧此失彼」的問題。改良二：對比限制——設定 clipLimit 把直方圖中過高的長條削頂，並把超出部分均攤到其他灰階，避免平坦區的雜訊被暴力放大。名稱 Contrast Limited 正來自這個削頂機制。*

6. 對彩色影像的 B、G、R 三通道分別做直方圖等化，會產生什麼問題?正確做法為何?

   *提示與參考解答：三個通道的映射函數各不相同，原本的色彩比例會被破壞，產生詭異的色偏。正確做法：轉到亮度與色彩分離的色彩空間(LAB 或 YCbCr)，只對亮度通道(L 或 Y)做等化或 CLAHE，色彩通道保持不變，再轉回 BGR。第 8 章會說明這些色彩空間的設計理由。*

7. 卷積與相關的差別是什麼?什麼情況下兩者結果相同?CNN 的卷積層與本章的空間濾波有何異同?

   *提示與參考解答：卷積先把核旋轉 180 度再做對應相乘相加，相關則直接相乘相加。對稱核(如高斯)兩者結果相同。理論上要求卷積是因為它滿足交換律與結合律，並對應第 6 章的卷積定理;OpenCV 的 filter2D 實際執行的是相關。CNN 卷積層的數學運算與本章相同，關鍵差異在於：本章的核是人設計的(高斯、Sobel),CNN 的核是從資料中學出來的(第 15 章)。*

8. 一個 11×11 的可分離高斯核，相對於直接二維卷積，每個像素可省下多少次乘法?這個技巧為什麼對第 17 章重要?

   *提示與參考解答：二維卷積每像素需 11×11 = 121 次乘法;可分離則先水平再垂直各做一維，需 11 + 11 = 22 次，省下 99 次，約快 5.5 倍。核愈大差距愈懸殊(51×51 時從 2601 降為 102)。對第 17 章重要，是因為邊緣裝置(Jetson、ESP32)算力有限，即時處理必須錙銖必較;而當核大到連可分離都不划算時，就該考慮轉到第 6 章的頻率域。*

9. 影像同時受到椒鹽雜訊污染且需保留邊緣，你會選哪一種平滑濾波?為什麼中值濾波對椒鹽近乎完美，對高斯雜訊卻無特別優勢?

   *提示與參考解答：選中值濾波。因為椒鹽雜訊是極端值(0 或 255)，排序後落在最前或最後，永遠不會被選為中位數，故能完全去除且不模糊邊緣。對高斯雜訊則無特別優勢：高斯雜訊是圍繞真值的小幅隨機起伏，並非極端值，取中位數與取平均的效果相近，而均值濾波在此情況下反而是統計上的最佳線性估計。*

10. 說明 Unsharp Masking 的公式與步驟。為什麼銳化應該在去雜訊之後執行?過度銳化會出現什麼現象?銳化有沒有真正「找回」被模糊掉的資訊?

*提示與參考解答：公式 g = f + k·(f - blur(f))：先模糊、原圖減模糊得細節層(高頻)、再加權加回。順序理由：高頻成分中雜訊佔比重，銳化會放大雜訊;若先銳化再去雜訊，雜訊已被放大，再平滑只會連同細節一起抹掉。過度銳化出現過衝(邊緣兩側的白邊黑邊，俗稱光暈，與第 6 章振鈴同源)。銳化並未找回資訊，它只是把邊緣兩側的對比拉大製造「更清楚」的視覺印象;真正的還原是第 7 章的反卷積。*

11. 為什麼多影格平均能降低雜訊?平均 25 張後雜訊標準差降為原來的幾分之一?它與空間平滑相比有什麼根本優勢?

    *提示與參考解答：若雜訊隨機且零均值，平均 N 張後訊號不變、雜訊正負相消，標準差降為原來的 N 平方根分之一;平均 25 張則降為五分之一。根本優勢：空間平滑平均的是「同一時刻的不同像素」，而鄰近像素的訊號未必相同(邊緣兩側就不同)，故會模糊細節;多影格平均的是「不同時刻的同一像素」，訊號理論上完全相同，因此能在幾乎不損失細節的前提下降噪。*

12. (綜合題)承叫獸開講：請說明 Night Sight 用到了本章與本書哪些技術，並解釋為什麼「對齊」是多影格方案的真正難點。

    *提示與參考解答：本章技術：多影格平均去雜訊(5.6 節)、Gamma 與色調曲線提亮(5.1 節)、局部對比增強(5.2 節)、銳化補回細節(5.5 節)。其他章節：對齊用第 6.7.2 節相位相關法或第 13 章特徵匹配、第 4 章的幾何轉換做補償、第 3 章的 PSNR/SSIM 評估、第 8 章的白平衡(Night Sight 用學習式演算法)。對齊是難點的原因：手持必有位移旋轉，不對齊直接平均會得到模糊照片;而場景中可能有移動的人車，強行整體對齊會產生鬼影，故 Night Sight 採用分塊對齊並丟棄運動過大的區塊(robust merge)。*

13. 承圖 5-1：單點、鄰域與全域運算的差別在於「輸出像素看到多少輸入」。直方圖等化為什麼歸類為全域運算而不是單點運算?CLAHE 又該歸在哪一類?

    *提示與參考解答：直方圖等化在「套用映射表」的那一步確實是逐像素進行的，但那張映射表本身來自整張影像的累積分布函數——換一張影像，表就不同。決定映射的資訊來自全域，因此歸為全域運算。CLAHE 介於兩者之間：它把影像切成小塊，每塊各自依自己的直方圖建表，所以是「局部全域」運算;若把一個分塊視為一個鄰域，也可以理解為大尺度的鄰域運算。這個分類的實益在於判斷「同樣的參數換一張影像還能不能用」——單點運算可以，全域運算不行。*

14. 承圖 5-2：對數轉換與 γ < 1 的 Gamma 曲線都能提亮暗部，兩者的差別在哪裡?什麼場合非用對數不可?

    *提示與參考解答：兩者的曲線都是上凸的，但對數在靠近 0 的地方斜率趨於無限大，對極暗的值提升極為劇烈;Gamma 的斜率雖大卻有限，提亮較為溫和可控。因此一般影像的提亮多用 Gamma(可用一個參數連續調整)，而當輸入的動態範圍橫跨數個數量級時就非用對數不可——最典型的是第 6 章的傅立葉頻譜，直流分量往往比高頻大上數萬倍，不取對數的話整張頻譜只會看到中央一個亮點。*

15. 計算題：3×3 高斯核為 (1/16)·[[1,2,1],[2,4,2],[1,2,1]]。某影像區塊的灰階為 [[120,140,135],[128,200,142],[118,133,126]]，請計算卷積後輸出的中心像素值。並說明核內權重總和為什麼必須是 1。

    *提示與參考解答：逐項相乘後相加：(120×1 + 140×2 + 135×1 + 128×2 + 200×4 + 142×2 + 118×1 + 133×2 + 126×1) / 16 = 2385 / 16 = 149.06，四捨五入為 149。權重總和必須為 1 的理由是「亮度守恆」：若總和大於 1，整張影像會整體變亮;小於 1 則整體變暗。對均值核與高斯核這類平滑濾波，我們只想重新分配鄰域內的能量、不想改變平均亮度，所以要正規化。反之，微分型的核(如 Sobel、Laplacian)權重總和為 0，因為它們量測的是「變化量」，平坦區應該輸出 0。*

16. 承圖 5-7：若濾波核改成 Sobel 的 [[−1,0,1],[−2,0,2],[−1,0,1]]，相關與卷積的結果差在哪裡?為什麼在深度學習的文獻中這兩個詞常被混用卻不出問題?

    *提示與參考解答：這個核左右反對稱，翻轉 180° 後正負號整個顛倒，因此相關與卷積的輸出互為相反數——梯度的方向會相反，若據此判斷「由亮到暗」或「由暗到亮」就會完全弄錯。深度學習不出問題的原因是：CNN 的核是「學出來的」而不是人訂的，框架實際做的是相關運算，網路只要在訓練中把核學成翻轉後的樣子，效果完全等價。換句話說差異被訓練吸收了，所以名詞混用無害;但在本章這種「核由人指定」的場合就必須分清楚。*

17. 承圖 5-9：為什麼補零(constant)會讓邊界變暗?對一張 512×512 的影像使用 31×31 的核，有多少比例的像素會受到邊界處理方式的影響?(請計算)

    *提示與參考解答：補零等於假設影像之外一片漆黑。靠近邊界的像素，其鄰域有一部分落在影像外並以 0 參與平均，拉低了結果，因此邊界形成一圈暗帶。計算：31×31 的核半徑為 15，只要像素距離任一邊界不足 15 就會受影響。不受影響的區*

*域為 (512 − 30)² = 482² = 232,324 像素;全圖 512² = 262,144 像素;受影響者 29,820 像素，佔 11.4%。核愈大、影像愈小，這個比例愈高——這也是為什麼在小影像或 ESP32-CAM 的低解析度影像上，邊界處理的選擇特別重要。*

18. 計算題：承圖 5-10。請分別計算 k = 5、11、21、31 的核，以二維卷積與可分離實作的每像素乘法次數與加速倍率。為什麼核再更大時，改到頻率域做會更划算?

    *提示與參考解答：二維為 k² 次、可分離為 2k 次，加速倍率 k/2：k=5 為 25 對 10(2.5 倍)、k=11 為 121 對 22(5.5 倍)、k=21 為 441 對 42(10.5 倍)、k=31 為 961 對 62(15.5 倍)。可分離實作的成本隨 k 線性成長，二維則是平方成長。但頻率域的成本幾乎與核大小無關——依卷積定理(第 6 章)，空間域的卷積等於頻率域的逐點相乘，成本由 FFT 決定，約為 O(MN log MN)。因此存在一個交叉點，核大於該點後頻率域勝出;OpenCV 的 filter2D 內部就會在核夠大時自動改用這條路徑。*

19. 承圖 5-11：為什麼雙邊濾波對椒鹽雜訊的表現反而最差(RMSE 31.0，比均值的 14.7 還糟)?請從它的值域權重解釋。

    *提示與參考解答：雙邊濾波的值域權重會把「灰階與中心差很多」的鄰居關掉，它的設計假設是「灰階差很大 = 跨過了邊緣 = 不該平均」。但椒鹽雜訊正是一群孤立的極端值(0 或 255)。當中心點本身就是一個椒鹽點時，所有正常鄰居都因灰階差太大而被關掉，幾乎只剩自己，於是那個錯誤的極端值被原封不動地保留下來。換句話說，雙邊濾波忠實地執行了「保邊」，只是它把雜訊誤認為邊緣。這提醒我們：濾波器的優劣取決於它的假設與實際雜訊型態是否相符。*

20. 承圖 5-12：雙邊濾波有空間權重與值域權重兩項。若把值域的標準差 σ_r 設得非常大(例如 1000)，雙邊濾波會退化成什麼?設得非常小又會如何?

    *提示與參考解答：σ_r 很大時，值域權重 exp(−Δ²/2σ_r²) 對任何灰階差都接近 1，等於沒有作用，雙邊濾波退化為單純的高斯濾波——保邊能力消失。反之 σ_r 很小時，只有灰階幾乎相同的鄰居才有權重，絕大多數鄰居被關掉，輸出幾乎等於輸入本身——濾波幾乎沒有發生，雜訊原封不動。因此 σ_r 應該設在「雜訊振幅」與「真實邊緣落差」之間：大於雜訊才壓得掉雜訊，小於邊緣落差才留得住邊緣。這也說明了為什麼雙邊濾波的參數需要依雜訊強度調整，不存在萬用值。*

21. 承圖 5-15：本章實測，對含雜訊的影像直接銳化會讓 RMSE 由 15.7 惡化到 30.8，先平滑再銳化則降到 10.2。請解釋這三個數字背後的機制。

    *提示與參考解答：15.7 是雜訊本身造成的誤差。直接銳化時，銳化的本質是「放大高頻」，而雜訊正是最典型的高頻成分，於是雜訊被連同細節一起放大，誤差幾乎翻倍到 30.8。先平滑再銳化則是先把高頻雜訊壓下去，再放大剩下的(以中低頻為主的)結構，誤差降到 10.2——比原本的雜訊影像還好，因為平滑本身就降低了誤差。結論是順序決定成敗：「先去雜訊，再銳化」。同樣的順序在第 9 章的 Canny(先高斯平滑再求梯度)與第 7 章的還原流程中都會再出現。*

22. 計算題：承圖 5-16。若單張影像的雜訊標準差為 24，要把雜訊降到 6 以下，至少需要平均幾張?若每張曝光 1/15 秒，總曝光時間是多久?這對手持拍攝意味著什麼?

    *提示與參考解答：多影格平均使雜訊以 $1/\sqrt{N}$ 下降，故需 $24/\sqrt{N} \le 6$，即 $\sqrt{N} \ge 4$、$N \ge 16$，至少 16 張。總曝光時間為 $16 \times (1/15) \approx 1.07$ 秒。對手持拍攝而言，一秒鐘已經遠超過人手能保持不動的時間(一般約 1/30 秒)，因此這 16 張之間必然存在數個到數十個像素的位移，不先對齊就直接平均只會得到一張模糊的照片(見圖 5-17)。這正是夜景模式的真正難點不在「平均」而在「對齊」的原因。*

23. 承圖 5-17：八張影格未對齊直接平均的 RMSE 為 18.9，對齊後為 9.5。為什麼未對齊的平均會「雜訊降了，但影像變糊」?這與空間平滑的效果有何異同?

    *提示與參考解答：平均仍然對每個像素位置累加了八個獨立的雜訊樣本，所以雜訊照樣以 $1/\sqrt{N}$ 下降——這部分有效。但因為影格之間有位移，同一個像素位置在不同影格中對應到場景的不同位置，於是相鄰的真實結構也被一起平均了，等同於做了一次空間模糊，訊號因此損失。與空間平滑的相同之處在於結果都是「模糊」;不同之處在於空間平滑的模糊程度由核大小決定、可控，而未對齊平均的模糊程度由手震幅度決定、不可控，且在畫面各處不一致。對齊之後，平均就只作用在雜訊上，這才是多影格相對於空間平滑的根本優勢。*

24. 承圖 5-18 與 5.7.1 節：工業前處理管線為「去雜訊 → 強化對比 → 銳化 → 偵測」。若把「銳化」提到「去雜訊」之前，結果會如何?若把「強化對比」整個拿掉，缺陷偵測會遇到什麼困難?

> *提示與參考解答：銳化提前會把雜訊放大(見圖 5-15)，後續的去雜訊既要處理原有雜訊、又要處理被放大的雜訊，往往得加大核，結果連缺陷的細節也一併抹掉——兩頭落空。拿掉強化對比則會遇到打光不均的問題：工件右側因為離光源遠而偏暗，若用一個全域門檻去找亮線缺陷，左側會因為整體偏亮而大量誤判，右側的缺陷則因為整體偏暗而漏判。CLAHE 的作用正是把各處的局部對比拉到同一個水準，讓後續可以用一組參數處理整張影像。*

25. (綜合題)設計題：某產線的工件表面有方向性的拉絲紋理，缺陷是淺淺的凹痕，而且打光從左側來，導致右側偏暗。請設計一條前處理管線，說明每一步的選擇理由，以及你會如何驗證每一步真的有幫助。

> *提示與參考解答：本題無標準答案，評分重點在理由與驗證方式。一條合理的管線：(1)先處理打光不均——用 CLAHE 做局部對比均化，或估計背景打光場後相除;(2)去雜訊要選保邊的方法，因為凹痕本身就是淺的，均值或大核高斯會直接抹掉它，宜用雙邊或小核中值;(3)方向性紋理是干擾——可考慮沿紋理方向做異向性平滑，或改到頻率域用方向性帶阻濾波把紋理的週期成分濾掉(第 6 章);(4)凹痕在側光下通常呈現暗線，宜用型態學黑帽運算而非頂帽(第 10 章);(5)最後以面積與長寬比篩選候選區域。驗證方式最關鍵：每一步都要有量化指標，例如缺陷區與背景的對比(CNR)、良品的誤報率、不良品的漏檢率，並且在固定的一批標註樣本上逐步比較——「看起來比較清楚」不是工程上可接受的驗收標準。*

## 參考資料與延伸閱讀


- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 3(空間域強化之標準參考)。
- Levoy, M., & Pritch, Y., Night Sight: Seeing in the Dark on Pixel Phones, Google Research Blog, Nov. 2018.(本章叫獸開講之技術來源)
- Liba, O., et al., Handheld Mobile Photography in Very Low Light, ACM TOG, 2019.(Night Sight 完整論文;運動測光與學習式白平衡)
- Hasinoff, S. W., et al., Burst Photography for High Dynamic Range and Low-Light Imaging on Mobile Cameras, ACM TOG, 2016.(HDR+ 原始論文)

- Zuiderveld, K., Contrast Limited Adaptive Histogram Equalization, Graphics Gems IV, 1994.(CLAHE 原始文獻)
- Tomasi, C., & Manduchi, R., Bilateral Filtering for Gray and Color Images, ICCV, 1998.


- 補充教材與課程資訊：@prof(https://page.line.me/prof)